

Graph Traversal Algorithms — Advanced Module

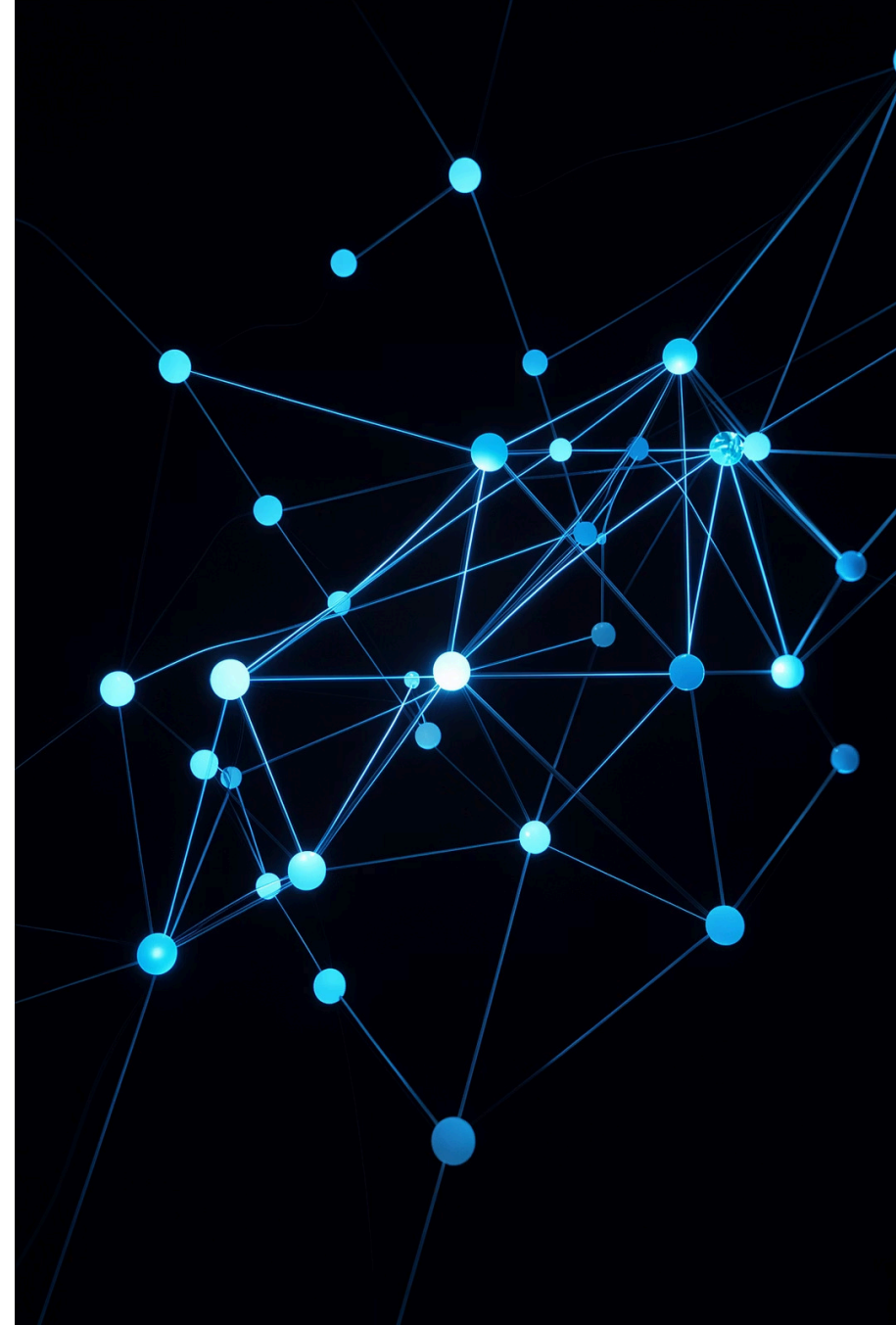
A deep dive into the two foundational graph traversal algorithms powering everything from GPS routing to compiler analysis.

Part 1: Breadth-First Search (BFS)

Shortest paths, level-order traversal, multi-source and bidirectional variants

Part 2: Depth-First Search (DFS)

Structural analysis, cycle detection, SCCs, bridges, and backtracking



PART 1: Breadth-First Search (BFS)

TOPIC CLASSIFICATION

Field	Details
Topic Name	Breadth-First Search (BFS)
Category	Graph Algorithms
Domain / Subdomain	Computer Science / Graph Traversal / Shortest Path
Topic Type	Algorithmic + Implementation + Problem Pattern
Difficulty Level	Advanced
Prerequisites	Graph Basics (adjacency list, directed/undirected), Queue data structure, Matrix as Graph
Industry Relevance	Critical — Shortest path in unweighted graphs, web crawlers, social network distance, network broadcast, GPS systems
Recommended Learning Path	Graph Basics → Matrix as Graph → BFS → DFS → Dijkstra → Bellman-Ford

What Is BFS?

Formal Definition

Breadth-First Search (BFS) is a graph traversal algorithm that explores all vertices at the current depth level before moving to vertices at the next depth level. It operates by maintaining a **FIFO queue** of discovered-but-unprocessed vertices, ensuring that vertices closer to the source are always processed before vertices further away.

The Core Guarantee: BFS on an unweighted graph produces a BFS tree in which the path from the source to any vertex is the shortest path (minimum number of edges).

This guarantee is the defining property of BFS and the reason it is the algorithm of choice for shortest-path problems on unweighted graphs and grids.

Why a Queue and Not a Stack?

Queue (FIFO) → BFS:

Process vertices by
DISTANCE from source

Level 0: [A]

Level 1: [B, C]

Level 2: [D, E, F]

Stack (LIFO) → DFS:

Process vertices by
DEPTH of exploration

A → B → D → E (depth first)

A queue ensures **all** level-k vertices are processed before any level-(k+1) vertex — the key to shortest path.

Why Does BFS Exist?

THE FUNDAMENTAL PROBLEM BFS SOLVES

DFS vs BFS on Path Finding

DFS on an unweighted graph finds *a* path. **BFS** finds *the shortest* path.

✅ **BFS is the provably correct algorithm for unweighted SSSP (Single Source Shortest Path).**

BFS is the provably correct algorithm for unweighted SSSP (Single Source Shortest Path). Because BFS explores the direct neighbor at level 1 before going deep into longer chains at level 3, it always discovers the minimum-edge path first.

Graph: A — B — D



DFS from A: might find A→B→D (length 2)

might find A→C→D (length 2)

Both correct, both shortest here.

Graph: A — B — C — D



DFS from A: might find A→B→C→D (length 3)

BFS from A: finds A→D (length 1 direct edge)

Because BFS explores the direct neighbor D at level 1 before going deep into B→C→D at level 3.

How BFS Works — Core Algorithms

1

Standard BFS

```
FUNCTION bfs(graph, source):  
    visited = set containing source  
    queue = deque([source])  
    order = []  
    WHILE queue not empty:  
        vertex = queue.popleft()  
        order.append(vertex)  
        FOR each neighbor u of  
vertex:  
            IF u NOT in visited:  
                visited.add(u)  
                queue.append(u)  
    RETURN order  
// Time:  $O(V+E)$  Space:  $O(V)$ 
```

2

BFS with Distance Tracking

```
FUNCTION bfs_distances(graph,  
source):  
    dist = {source: 0}  
    parent = {source: None}  
    queue = deque([source])  
    WHILE queue not empty:  
        u = queue.popleft()  
        FOR each neighbor v of u:  
            IF v NOT in dist:  
                dist[v] = dist[u] + 1  
                parent[v] = u  
                queue.append(v)  
    RETURN dist, parent  
// dist[v] = shortest distance  
from source
```

3

Path Reconstruction

```
FUNCTION  
reconstruct_path(parent,  
source, target):  
    IF target NOT in parent:  
        RETURN [] // Target  
unreachable  
    path = []  
    current = target  
    WHILE current is not None:  
        path.append(current)  
        current = parent[current]  
    path.reverse()  
    RETURN path  
// parent=  
{A:None,B:A,C:A,D:B,E:C}  
// reconstruct(parent,A,D) →  
[A,B,D] ✓
```

BFS Execution & Advanced Variants

Level-by-Level Trace

Graph (undirected):



BFS from A:

Initial: queue=[A], visited={A}

Step 1: Dequeue A → Enqueue B,C

Step 2: Dequeue B → Enqueue D,E

Step 3: Dequeue C → Enqueue F

Step 4: Dequeue D → no new neighbors

Step 5: Dequeue E → Enqueue G

Step 6: Dequeue F → done

Step 7: Dequeue G → done

BFS order: A, B, C, D, E, F, G

Distances: A=0,B=1,C=1,D=2,E=2,F=2,G=3

← Guaranteed shortest distances ✓

Multi-Source BFS

```
FUNCTION multi_source_bfs(graph, sources):
```

```
    dist = {}
```

```
    queue = deque()
```

```
    FOR s in sources:
```

```
        dist[s] = 0
```

```
        queue.append(s)
```

```
    WHILE queue not empty:
```

```
        u = queue.popleft()
```

```
        FOR v in graph.neighbors(u):
```

```
            IF v NOT in dist:
```

```
                dist[v] = dist[u] + 1
```

```
                queue.append(v)
```

```
    RETURN dist
```

```
// dist[v] = shortest distance from ANY source
```

```
// All sources start at level 0
```

0-1 BFS (Binary Edge Weights)

```
FOR (v, weight) in graph.neighbors(u):
```

```
    IF dist[u] + weight < dist[v]:
```

```
        dist[v] = dist[u] + weight
```

```
        IF weight == 0:
```

```
            deque.appendleft(v) // Free edge → front
```

```
        ELSE:
```

```
            deque.append(v) // Cost edge → back
```

```
// O(V+E) instead of O((V+E) log V)
```

Bidirectional BFS

For shortest path between a specific source and target, bidirectional BFS runs two simultaneous BFS searches — one from source, one from target — and terminates when they meet. This reduces the search space from **$O(b^d)$** to **$O(b^{(d/2)})$** where b = branching factor, d = path length. Used in Google Maps (bidirectional Dijkstra) and LinkedIn's 6-degrees-of-separation feature.

BFS Variants — Quick Reference

Variant	When to Use	Key Difference
Standard BFS	Traversal / reachability	Basic queue
BFS with distance	Shortest path (unweighted)	Track dist[] array
Multi-source BFS	Nearest-source distance	All sources in queue at dist=0
Bidirectional BFS	Known source AND target	Two queues; meet in middle
0-1 BFS	Binary edge weights (0 or 1)	Deque; front for 0-edges
Level-order BFS	Level-by-level processing	Track level_size
BFS on implicit graph	State-space search	Generate neighbors on-the-fly

Real-World Use Cases — BFS

USE CASE 1

Web Crawler — Crawl Depth Limiting

Context: Search engines (Googlebot, Bingbot)

Problem: Starting from a seed URL, crawl all reachable pages.
Limit crawl to depth $\leq K$.

Solution: BFS where pages = vertices, hyperlinks = edges.
Process level by level. Stop after K levels.

 BFS FIFO ensures pages are crawled in distance order from seed. This prevents going deep into one link chain while missing closely linked pages.

Level 0: seed URL

Level 1: all URLs linked from seed

Level 2: all URLs linked from
level-1 pages

...

Level K: stop

BFS FIFO ensures pages are crawled
in distance order from seed.

This prevents going deep into one
link chain while missing closely
linked pages.

BFS in Social Networks & GPS



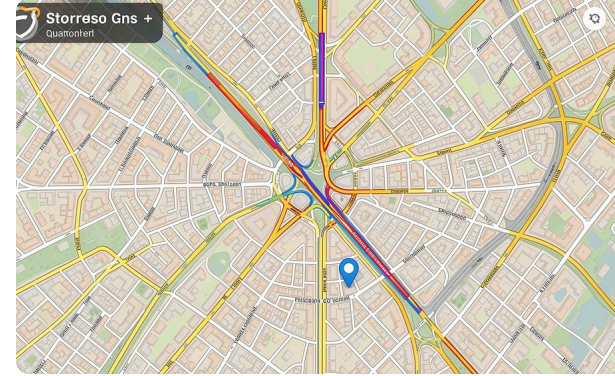
Social Network — Six Degrees of Separation

Context: LinkedIn "Degrees of Separation", Facebook friend suggestions

Problem: Are two users within 3 degrees of separation? Find all users within 2 degrees.

Solution: BFS from User A. Track depth; stop when reaching User B or depth limit.

LinkedIn's actual implementation: Bidirectional BFS — search from A and B simultaneously. For their graph of 900M+ nodes, this is the difference between searching millions of nodes vs. thousands.



GPS / Mapping — Shortest Route

Context: Google Maps, Apple Maps, Uber routing

Problem: Find shortest driving route from A to B (unweighted: fewest turns; weighted: minimum time).

Solution: BFS for unweighted (fewest roads traversed). Bidirectional Dijkstra (= BFS with priority queue) for weighted.

Production detail: At Google Maps scale, Contraction Hierarchies (preprocessing-based Dijkstra) is used. But the conceptual core is BFS-derived.

Network Broadcast & Spanning Tree

Use Case 4: Network Broadcast

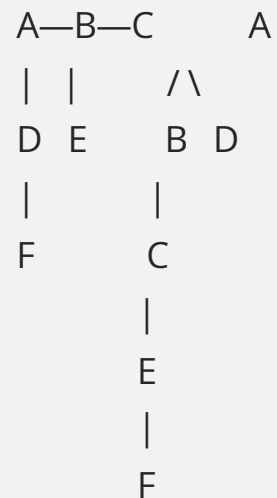
Context: Computer networks, Ethernet flooding, distributed systems

Problem: Broadcast a message from one router to all routers; build a spanning tree to avoid loops.

Solution: BFS from the root router naturally builds a BFS spanning tree. The BFS tree becomes the broadcast tree; packets only forwarded on tree edges.

- ✓ Spanning tree avoids loops; BFS tree = minimum-depth broadcast tree

Network: BFS tree (broadcast paths):



Spanning tree avoids loops;
BFS tree = minimum-depth
broadcast tree

BFS SSSP with Path Reconstruction

IMPLEMENTATION 1

```
FUNCTION bfs_sssp(graph, source, target, V):
    dist = array of -1, size V
    parent = array of -1, size V
    dist[source] = 0
    queue = deque([source])

    WHILE queue not empty:
        u = queue.popleft()
        IF u == target: BREAK // Early exit once target reached
        FOR v in graph.neighbors(u):
            IF dist[v] == -1: // Unvisited
                dist[v] = dist[u] + 1
                parent[v] = u
                queue.append(v)

    // Reconstruct path
    IF dist[target] == -1: RETURN -1, [] // Unreachable

    path = []
    cur = target
    WHILE cur != -1:
        path.append(cur)
        cur = parent[cur]
    RETURN dist[target], path[::-1] // Reverse to get source→target order
```

- ❏ Always maintain a **parent[]** array alongside **dist[]** to enable $O(\text{path_length})$ path reconstruction. Without **parent[]**, you can report the distance but not the actual path.

Check if Bipartite via BFS

IMPLEMENTATION 2

Concept

A graph is bipartite if and only if it has no odd-length cycle, which is equivalent to being 2-colorable.

BFS approach: Color source with 0, all neighbors with 1, their neighbors with 0... If we ever try to color a vertex a color it already has → not bipartite.

 Works on disconnected graphs — the outer loop handles each connected component independently.
Time: $O(V + E)$

```
FUNCTION is_bipartite(graph, V):  
    color = array of -1, size V  
  
    FOR start = 0 TO V-1:  
        IF color[start] != -1: CONTINUE  
        color[start] = 0  
        queue = deque([start])  
  
        WHILE queue not empty:  
            u = queue.popleft()  
            FOR v in graph.neighbors(u):  
                IF color[v] == -1:  
                    color[v] = 1 - color[u]  
                    queue.append(v)  
                ELIF color[v] == color[u]:  
                    RETURN FALSE // Same color!  
  
    RETURN TRUE
```

Level-Order BFS with Level Separation

IMPLEMENTATION 3

```
// Returns list of lists — each inner list = vertices at same BFS level.  
// Application: tree level-order print, minimum depth of BST leaf, etc.
```

```
FUNCTION bfs_by_level(graph, source):
```

```
    visited = {source}
```

```
    queue = deque([source])
```

```
    levels = []
```

```
    WHILE queue not empty:
```

```
        level_size = len(queue)    // All vertices currently in queue = same level
```

```
        current_level = []
```

```
        REPEAT level_size times:
```

```
            u = queue.popleft()
```

```
            current_level.append(u)
```

```
            FOR v in graph.neighbors(u):
```

```
                IF v NOT in visited:
```

```
                    visited.add(v)
```

```
                    queue.append(v)
```

```
        levels.append(current_level)
```

```
    RETURN levels
```

```
// Key: snapshot queue size at start of each level iteration.
```

```
// All items enqueued during a level belong to the NEXT level.
```

📌 **Key insight:** Snapshot the queue size at the start of each level iteration. All items enqueued during processing of a level belong to the *next* level — never mix them.

Minimum Knight Moves — BFS on Implicit Graph

IMPLEMENTATION 4

Problem

Chess knight on an infinite board. Find the minimum number of moves from (0,0) to (x,y).

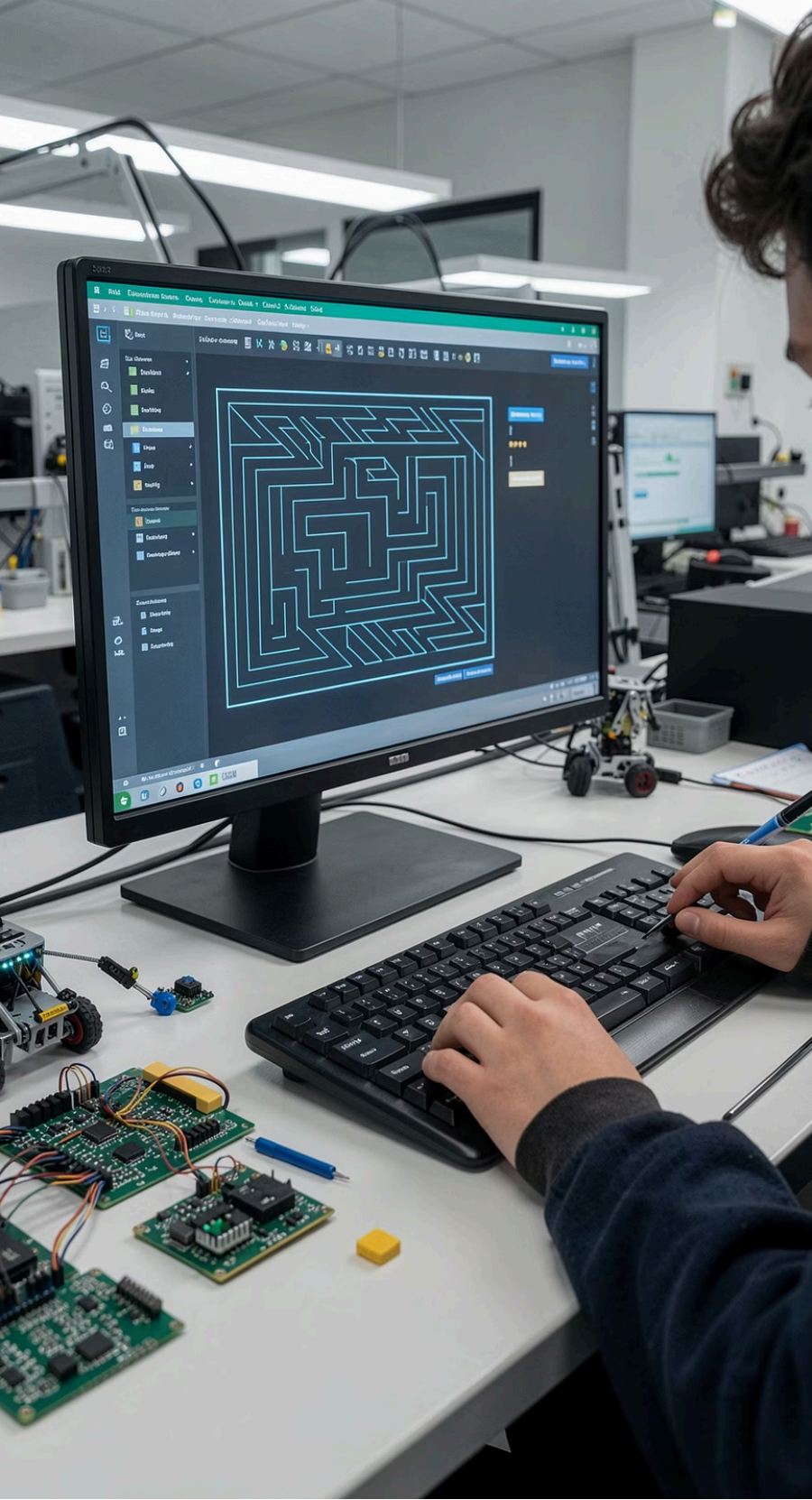
Each (r,c) coordinate is a vertex; edges are valid knight moves. BFS guarantees minimum steps.

Pruning: The target is in the first quadrant; don't go too far negative. This avoids exploring half the infinite board.

```
FUNCTION min_knight_moves(x, y):
    MOVES = [(-2,-1),(-2,1),(-1,-2),(-1,2),
             (1,-2),(1,2),(2,-1),(2,1)]
    target = (abs(x), abs(y)) // Symmetry
    queue = deque([(0, 0, 0)])
    visited = {(0, 0)}

    WHILE queue not empty:
        (r, c, steps) = queue.popleft()
        IF (r, c) == target: RETURN steps
        FOR (dr, dc) in MOVES:
            nr = r + dr; nc = c + dc
            IF (nr, nc) NOT in visited
                AND nr >= -1 AND nc >= -1:
                    visited.add((nr, nc))
                    queue.append((nr,nc,steps+1))

    // BFS guarantees minimum steps.
    // Pruning avoids exploring half the
    // infinite board.
```

Hands-On Lab 1: BFS Shortest Path with Multiple Targets

ADVANCED LAB

Objective

Find the shortest path from source to the **nearest** of multiple targets.

Setup

0—1—2

| |

3 5

| |

4—6—7

Source: 0

Targets: {4, 7}

Tasks

1. Implement BFS with distance tracking
2. Terminate as soon as ANY target is dequeued
3. Reconstruct the path using parent array
4. Verify: distances to 4 and 7 are both 3 (paths: $0 \rightarrow 3 \rightarrow 4$ and $0 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 7$)

Extension

Reverse the problem — which vertex in the graph is farthest from vertex 0? Run full BFS and find the maximum distance.

Hands-On Lab 2: BFS State-Space Search — Sliding Puzzle

ADVANCED LAB

Objective

Solve the 8-puzzle (3×3 grid, tiles 1-8 + blank) using BFS. Each grid configuration = a vertex. Valid tile moves = edges. BFS finds minimum moves.

State: tuple of 9 integers (flattened 3×3)

Start: (1,2,3,4,5,6,7,0,8) 0 = blank

Goal: (1,2,3,4,5,6,7,8,0)

Example move:

1 2 3 1 2 3

4 5 6 → 4 5 6

7 _ 8 7 8 _

(slide 8 left into blank)

Steps

1. Represent state as tuple (hashable for visited set)
2. Generate neighbors by finding blank (0), computing valid swap positions
3. Run BFS from start state to goal state
4. Count BFS levels = minimum moves



Insight: The 8-puzzle has $9!/2 = 181,440$ reachable states — tractable for BFS. The 15-puzzle (4×4) has 10.5 trillion states — BFS is infeasible; A* with admissible heuristic needed.

BFS Performance Considerations

Variant	Time	Space	Notes
Standard BFS	$O(V+E)$	$O(V)$	Each vertex/edge processed once
BFS on grid	$O(\text{ROWS} \times \text{COLS})$	$O(\text{ROWS} \times \text{COLS})$	$V = \text{cells}$, $E = \text{adjacencies}$
Multi-source BFS	$O(V+E)$	$O(V)$	All sources at level 0
Bidirectional BFS	$O(b^{(d/2)})$	$O(b^{(d/2)})$	$b = \text{branching factor}$, $d = \text{distance}$
0-1 BFS	$O(V+E)$	$O(V)$	Deque operations are $O(1)$
BFS on implicit graph	$O(\text{States} + \text{Transitions})$	$O(\text{States})$	State generation may be expensive

The "Mark on Enqueue" Rule — Complexity Impact

```
// Mark on DEQUEUE (WRONG): same vertex can be enqueued O(degree) times
// For dense graphs:  $O(E^2/V)$  — catastrophic

// Mark on ENQUEUE (CORRECT):
visited[v] = TRUE      // Before appending
queue.append(v)        // Guaranteed: each vertex enqueued at most once
// Total work:  $O(V + E)$  ← provably correct
```

BFS Best Practices & Common Mistakes

✓

DO'S

Category	Practice
Correctness	Mark vertices as visited when ENQUEUEING — prevents duplicate processing
Design	Use deque for O(1) popleft; regular arrays give O(n) popleft
Shortest Path	For unweighted SSSP, always use BFS — never DFS
Implementation	Use parent[] array alongside dist[] for path reconstruction
Performance	Consider bidirectional BFS when source and target are both known

✗

DON'TS

Category	Anti-Pattern
Correctness	Don't mark visited on dequeue — leads to exponential revisiting
Performance	Don't use BFS for weighted graphs — ignores weights, gives wrong distances
Implementation	Don't use list with .pop(0) — O(n) per pop = O(n²) total
Design	Don't restart BFS for each source in multi-source — initialize all together

Level	Mistake	Impact	Fix
Intermediate	Using list.pop(0) instead of deque	O(n²) total	Use collections.deque with .popleft()
Intermediate	Marking visited on dequeue	Same vertex enqueued O(degree) times	Mark visited immediately before .append()
Advanced	Using BFS for weighted shortest path	Wrong distances returned	Use Dijkstra for weighted graphs
Advanced	Not tracking parent for path reconstruction	Can report distance but not the path	Always maintain parent[] alongside dist[]

BFS Learning Summary

1. Core Structure

FIFO queue; guaranteed shortest path (minimum edges) on unweighted graphs

2. Mark on Enqueue

Mark vertices as visited when enqueueing — not dequeuing. Prevents $O(V^2)$ blow-up

3. Use deque

$O(1)$ popleft; list.pop(0) is $O(n)$ — a critical performance difference

4. Multi-Source

All sources at distance 0 in queue simultaneously — spreads like a wave

5. Bidirectional

Two BFS searches from source and target; meet in middle $\rightarrow O(b^{(d/2)})$

6. 0-1 BFS

Deque; push to front for 0-weight edges, back for 1-weight $\rightarrow O(V+E)$

7. Implicit Graphs

Generate neighbors on-the-fly from state transitions (word ladder, knight moves)

BFS QUICK REFERENCE

STRUCTURE: FIFO Queue | Processes nearest nodes first

GUARANTEE: Shortest path on UNWEIGHTED graphs

TIME: $O(V + E)$ | SPACE: $O(V)$

TEMPLATE:

```
dist[source] = 0
queue = deque([source])
WHILE queue:
    u = queue.popleft()
    FOR v in neighbors(u):
        IF v not visited:
            dist[v] = dist[u] + 1
            visited.add(v) ← MARK BEFORE APPEND
            queue.append(v)
```

USE BFS WHEN:

Shortest path, unweighted | Connected components
Level-order processing | State-space minimum moves
Nearest source distance | Bipartite check

PART 2: Depth-First Search (DFS)

TOPIC CLASSIFICATION

Field	Details
Topic Name	Depth-First Search (DFS)
Category	Graph Algorithms
Domain / Subdomain	Computer Science / Graph Traversal / Structural Analysis
Topic Type	Algorithmic + Implementation + Problem Pattern
Difficulty Level	Advanced
Prerequisites	Graph Basics, BFS, Recursion, Stack data structure
Industry Relevance	Critical — Cycle detection, topological sort, SCC, articulation points, maze solving, backtracking, compiler analysis, game tree search
Recommended Learning Path	Graph Basics → BFS → DFS → Topological Sort → SCC → Bridges & Articulation Points

What Is DFS?

Formal Definition

Depth-First Search (DFS) is a graph traversal algorithm that explores each branch of the graph as far as possible before backtracking. It operates using a **LIFO stack** (either the call stack via recursion, or an explicit stack), exploring one neighbor completely before moving to the next.

Discovery and Finish Times

Assign each vertex a **discovery time** (when first visited) and **finish time** (when DFS from it completes). These timestamps are central to Tarjan's SCC, topological sort, and bridge detection.

Key property: For any tree edge ($u \rightarrow v$): $\text{disc}[u] < \text{disc}[v] < \text{finish}[v] < \text{finish}[u]$ — "v's entire DFS is contained within u's DFS"

DFS Edge Classification

During DFS from vertex u to vertex v :

TREE EDGE: v not yet visited $\rightarrow$ DFS recurses
Forms the DFS spanning tree

BACK EDGE: v is an ancestor of u in DFS tree
DIRECTED graph: back edge = cycle!
UNDIRECTED: the "parent" edge

FORWARD EDGE: v is a descendant of u , not via direct tree edge
Only in DIRECTED graphs

CROSS EDGE: v is neither ancestor nor descendant
Only in DIRECTED graphs

DFS Tree:

A (start, DFS #1)

/ \

B C (DFS #2, #4)

/ \ \

D E F (DFS #3, #5, #6)

|

A $\leftarrow$ Back edge! ($E \rightarrow A$ = CYCLE)

How DFS Works — Core Algorithms

1

Recursive DFS

```
FUNCTION dfs_recursive(graph,
u, visited):
    visited[u] = TRUE
    FOR v in graph.neighbors(u):
        IF NOT visited[v]:
            dfs_recursive(graph, v,
visited)

// Time: O(V+E)
// Space: O(V) recursion stack
// Simple, clean — backbone of
most
// DFS applications
```

2

Iterative DFS (Explicit Stack)

```
FUNCTION dfs_iterative(graph,
source):
    visited = set()
    stack = [source]
    order = []
    WHILE stack not empty:
        u = stack.pop()    // LIFO
        IF u in visited: CONTINUE
        visited.add(u)
        order.append(u)
        FOR v in
reversed(graph.neighbors(u)):
            IF v NOT in visited:
                stack.append(v)
    RETURN order
// Mark visited AFTER pop (not
before push)
```

3

BFS vs DFS Structural Difference

Graph: A—B—D
|
C—E

BFS from A:	DFS from A:
Level 0: A (backtrack)	A→B→D
Level 1: B, C (backtrack)	A→C→E
Level 2: D, E	

BFS order: A,B,C,D,E
DFS order: A,B,D,C,E

BFS: shortest path
DFS: structural analysis

Advanced DFS: Topological Sort & Tarjan's SCC

Topological Sort (DFS-Based)

Run DFS; when a vertex's DFS **finishes** (all descendants processed), prepend it to the result list. The resulting list is a valid topological ordering.

```
FUNCTION topological_sort_dfs(graph, V):
    visited = array of FALSE, size V
    result = []

    FUNCTION dfs(u):
        visited[u] = TRUE
        FOR v in graph.neighbors(u):
            IF NOT visited[v]:
                dfs(v)
        result.append(u) // After all descendants

    FOR u = 0 TO V-1:
        IF NOT visited[u]:
            dfs(u)

    RETURN result[::-1] // Reverse finish order
```

```
// Why reverse? A node finishes AFTER all its
// dependencies. Reversing gives:
// dependencies before dependents. ✓
```

Tarjan's SCC — Key Concept

A **Strongly Connected Component (SCC)** is a maximal subset of vertices where every vertex can reach every other vertex. Tarjan's finds all SCCs in a single DFS pass using `disc[]` and `low[]`.

```
FUNCTION tarjan_scc(graph, V):
    disc, low, on_stack = arrays
    stack = []; timer = [0]; sccs = []

    FUNCTION dfs(u):
        disc[u] = low[u] = timer[0]++
        stack.append(u); on_stack[u] = TRUE
        FOR v in graph.neighbors(u):
            IF disc[v] == -1:
                dfs(v)
                low[u] = MIN(low[u], low[v])
            ELIF on_stack[v]:
                low[u] = MIN(low[u], disc[v])
        IF low[u] == disc[u]: // SCC root!
            scc = []
            WHILE TRUE:
                w = stack.pop()
                on_stack[w] = FALSE
                scc.append(w)
                IF w == u: BREAK
            sccs.append(scc)
```

```
// Time: O(V+E) — single DFS pass
```

Tarjan's SCC — Full Walkthrough

Example Graph

Directed graph:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ ← cycle: SCC $\{0,1,2\}$

$2 \rightarrow 3$

$3 \rightarrow 4 \rightarrow 3$ ← cycle: SCC $\{3,4\}$

$4 \rightarrow 5$ ← SCC $\{5\}$ (single vertex)

✅ Result: SCCs = $\{5\}, \{4,3\}, \{2,1,0\}$

Step-by-Step Trace

Tarjan DFS from 0:

$\text{disc}[0]=0, \text{low}[0]=0, \text{stack}=[0]$

→ visit 1: $\text{disc}[1]=1, \text{stack}=[0,1]$

→ visit 2: $\text{disc}[2]=2, \text{stack}=[0,1,2]$

→ 0 on stack: $\text{low}[2]=\min(2,0)=0$

→ visit 3: $\text{disc}[3]=3, \text{stack}=[\dots,3]$

→ visit 4: $\text{disc}[4]=4, \text{stack}=[\dots,4]$

→ 3 on stack: $\text{low}[4]=\min(4,3)=3$

→ visit 5: $\text{disc}[5]=5$

$\text{low}[5]==\text{disc}[5] \rightarrow \text{SCC: } \{5\}$

$\text{low}[4]=3; \text{low}[4] \neq \text{disc}[4] \rightarrow \text{not root}$

$\text{low}[3]=\min(3, \text{low}[4]=3)=3$

$\text{low}[3]==\text{disc}[3] \rightarrow \text{SCC ROOT: } \{4,3\}$

$\text{low}[2]=\min(0, \text{low}[3]=3)=0$

$\text{low}[2] \neq \text{disc}[2] \rightarrow \text{not root}$

$\text{low}[1]=\min(1, \text{low}[2]=0)=0$

$\text{low}[1] \neq \text{disc}[1] \rightarrow \text{not root}$

$\text{low}[0]=0==\text{disc}[0] \rightarrow \text{SCC ROOT: } \{2,1,0\}$

Articulation Points and Bridges

Definitions

Articulation Point: A vertex whose removal disconnects the graph.

Bridge: An edge whose removal disconnects the graph.

Both are found in a single DFS pass using `disc[]` and `low[]` — the same arrays as Tarjan's SCC.

- ❏ **Bridge condition:** `low[v] > disc[u]` (strict greater than)
- Articulation point:** `low[v] >= disc[u]` (greater than OR equal)

```
FUNCTION find_bridges_and_APs(graph, V):
    disc, low, parent, visited = arrays
    bridges = []; art_points = set()

    FUNCTION dfs(u):
        visited[u] = TRUE
        disc[u] = low[u] = timer++
        child_count = 0
        FOR v in graph.neighbors(u):
            IF NOT visited[v]:
                child_count += 1
                parent[v] = u
                dfs(v)
                low[u] = MIN(low[u], low[v])
                // Articulation point:
                IF parent[u] == -1 AND child_count > 1:
                    art_points.add(u)
                IF parent[u] != -1 AND low[v] >= disc[u]:
                    art_points.add(u)
                // Bridge:
                IF low[v] > disc[u]:
                    bridges.append((u, v))
            ELIF v != parent[u]:
                low[u] = MIN(low[u], disc[v])

    dfs(0)

    // Time: O(V+E) Space: O(V)
```

DFS on DAG — Counting Paths & Backtracking

Counting Paths on a DAG

```
FUNCTION count_paths(graph, source, target):
```

```
    memo = {}
```

```
    FUNCTION dfs(u):
```

```
        IF u == target: RETURN 1
```

```
        IF u in memo: RETURN memo[u]
```

```
        total = 0
```

```
        FOR v in graph.neighbors(u):
```

```
            total += dfs(v)
```

```
        memo[u] = total
```

```
        RETURN total
```

```
    RETURN dfs(source)
```

```
// DFS + memoization on DAG =  $O(V + E)$ 
```

```
// Without memoization: exponential
```

```
// Foundation of DP on DAGs
```

Backtracking — N-Queens

```
FUNCTION solve_n_queens(n):
```

```
    results = []
```

```
    board = array of -1, size n
```

```
    FUNCTION backtrack(row):
```

```
        IF row == n:
```

```
            results.append(copy of board)
```

```
            RETURN
```

```
        FOR col = 0 TO n-1:
```

```
            IF is_safe(board, row, col):
```

```
                board[row] = col // Place queen
```

```
                backtrack(row + 1) // Recurse
```

```
                board[row] = -1 // BACKTRACK
```

```
    backtrack(0)
```

```
    RETURN results
```

```
// DFS explores all placements.
```

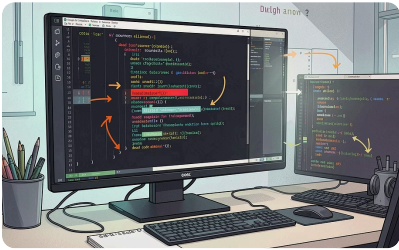
```
// Backtracking prunes invalid branches.
```

```
// No BFS equivalent for CSP search.
```

DFS Variants and Applications

DFS Variant	Core Mechanism	Application
Standard DFS	Recursive / iterative	Traversal, connectivity
DFS with timestamps	disc[] + finish[]	SCC, bridge detection, topo sort
DFS with rec_stack	Active path tracking	Directed cycle detection
DFS with low[]	Back-edge reachability	Bridges, articulation points, SCC (Tarjan)
DFS + memoization	Caching on DAG	Path counting, DAG DP
Backtracking DFS	Undo on backtrack	N-Queens, Sudoku, Word Search, subset generation
Iterative DFS	Explicit stack	Avoids stack overflow on deep graphs

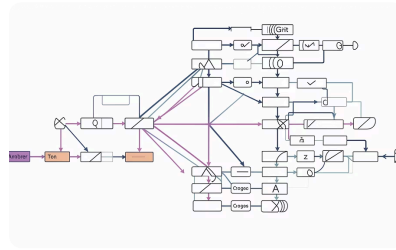
DFS Real-World Use Cases



Compiler — Dead Code Elimination

Context: GCC, Clang, LLVM

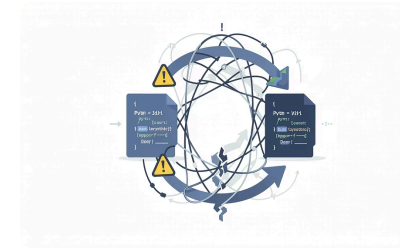
Build a call graph (functions = vertices; function calls = edges). DFS from `main`. Any unvisited vertex after DFS = dead code → removed from binary.



Git Merge Base — LCA on DAG

Context: Git version control

`git merge` and `git rebase` need the most recent common ancestor of two branches. Git's commit history is a DAG. DFS from both branch heads finds the first common vertex (LCA on DAG).



Circular Import Detection

Context: Python, JavaScript, Java build systems

Module dependency graph is directed. DFS with `rec_stack` detects back edges = circular imports. `module_a` → `module_b` → `module_c` → `module_a` → CIRCULAR IMPORT DETECTED.

DFS in Web Clustering & Game AI

Web Page Clustering — SCC

Context: Google PageRank analysis, link spam detection

Problem: Find groups of web pages that all link to each other (link farms). Each SCC = a "cluster" of mutually linking pages.

Solution: Tarjan's SCC via DFS. Pages within an SCC all link to each other → potential link farm. PageRank collapses SCCs into single super-nodes for computation efficiency.

Game AI — Minimax Tree Search

Context: Chess engines (Stockfish), Go engines (AlphaGo before NN)

Problem: Search the game tree to find the best move.

Solution: DFS on the game tree (implicit graph where states = vertices, valid moves = edges). Alpha-beta pruning cuts branches that cannot affect the final decision.

DFS explores game tree to depth D.

Backtracking returns minimax value.

Alpha-beta pruning prunes subtrees **where:**

maximizer found $\geq$ **beta** (opponent's bound)

minimizer found $\leq$ **alpha** (our bound)

DFS — All Paths from Source to Target

IMPLEMENTATION 1

```
FUNCTION all_paths(graph, source, target):
    results = []
    path = [source]

    FUNCTION dfs(u):
        IF u == target:
            results.append(copy of path)
            RETURN
        FOR v in graph.neighbors(u):
            IF v NOT in path:           // Avoid cycles (simple paths only)
                path.append(v)
                dfs(v)
            path.pop()                 // Backtrack

    dfs(source)
    RETURN results

// On DAGs: remove the "v NOT in path" check and use visited[] instead
// (guaranteed no cycles; simpler and faster)
```

- ❏ The `path.pop()` after the recursive call is the canonical backtracking pattern. Always undo state changes on return — forgetting this is the most common correctness bug in backtracking problems.

Kosaraju's SCC — Two-Pass DFS

IMPLEMENTATION 2

Algorithm

Pass 1: DFS on original graph; record finish order (push to stack when finished).

Pass 2: DFS on *reversed* graph in reverse finish order. Each DFS tree in pass 2 = one SCC.

📄 Kosaraju vs Tarjan: Kosaraju is cleaner conceptually (two simple DFS passes) — preferred in interviews. Tarjan is single-pass and more cache-friendly — preferred in production. Both are $O(V+E)$.

```
FUNCTION kosaraju_scc(graph, graph_rev, V):
    visited = array of FALSE, size V
    finish_stack = []

    FUNCTION dfs1(u):
        visited[u] = TRUE
        FOR v in graph.neighbors(u):
            IF NOT visited[v]: dfs1(v)
        finish_stack.append(u) // Push when done

    FOR u = 0 TO V-1:
        IF NOT visited[u]: dfs1(u)

    visited = array of FALSE, size V
    sccs = []

    FUNCTION dfs2(u, scc):
        visited[u] = TRUE
        scc.append(u)
        FOR v in graph_rev.neighbors(u):
            IF NOT visited[v]: dfs2(v, scc)

    WHILE finish_stack not empty:
        u = finish_stack.pop()
        IF NOT visited[u]:
            scc = []
            dfs2(u, scc)
            sccs.append(scc)

    RETURN sccs
```

Word Search — Backtracking DFS on Grid

IMPLEMENTATION 3

// Find if word **exists** in a 2D **character** grid **using** adjacent cells (**4-dir**, **no** reuse).

FUNCTION word_search(board, word):

ROWS = len(board); **COLS** = len(board[0])

FUNCTION dfs(r, c, index):

 IF index == len(word): **RETURN TRUE** // All characters matched

 IF r < 0 OR r >= **ROWS** OR c < 0 OR c >= **COLS**: **RETURN FALSE**

 IF board[r][c] != word[index]: **RETURN FALSE**

 temp = board[r][c]

 board[r][c] = '#' // Mark visited (**in-place**)

 found = (dfs(r-1, c, index+1) OR

 dfs(r+1, c, index+1) OR

 dfs(r, c-1, index+1) OR

 dfs(r, c+1, index+1))

 board[r][c] = temp // Restore (backtrack)

RETURN found

FOR r = 0 **TO** **ROWS**-1:

FOR c = 0 **TO** **COLS**-1:

 IF dfs(r, c, 0): **RETURN TRUE**

RETURN FALSE

// Backtracking: **in-place** mark + restore **is** the canonical pattern.

// **Time**: $O(\text{ROWS} \times \text{COLS} \times 4^L)$ where L = word length

Course Schedule — Cycle Detection as Feasibility Check

IMPLEMENTATION 4

Problem

Given N courses and prerequisites, can all courses be completed? Equivalent to: is the prerequisite DAG acyclic?

3-State DFS

```
FUNCTION can_finish(num_courses, prerequisites):
    graph = adjacency list, size num_courses
    FOR (a, b) in prerequisites:
        graph[b].append(a) // b before a

    NOT=0; PROGRESS=1; DONE=2
    state = array of NOT, size num_courses

    FUNCTION dfs(u):
        IF state[u] == PROGRESS:
            RETURN FALSE // Back edge = cycle
        IF state[u] == DONE:
            RETURN TRUE // Already processed

        state[u] = PROGRESS
        FOR v in graph[u]:
            IF NOT dfs(v): RETURN FALSE
        state[u] = DONE
        RETURN TRUE

    FOR u = 0 TO num_courses-1:
        IF NOT dfs(u): RETURN FALSE
    RETURN TRUE

// 3-state: cleaner than visited+rec_stack
// Time: O(V + E)
```

NOT_VISITED (0)

Not yet explored

IN_PROGRESS (1)

Currently in recursion stack

DONE (2)

Fully processed, no cycle

Hands-On Lab 1: DFS Timestamps and Edge Classification

ADVANCED LAB

Objective

Implement DFS with discovery/finish times; classify every edge.

Graph (directed)

```
0 → 1 → 3
0 → 2 → 4
1 → 2    ← cross edge
4 → 1    ← back edge (cycle!)
```

Edge Classification Rules

- **Tree Edge:** $\text{disc}[v] == -1$ at time of traversal
- **Back Edge:** $\text{disc}[v] != -1$ AND $\text{finish}[v] == -1$ (v in rec stack)
- **Forward Edge:** $\text{disc}[u] < \text{disc}[v]$ AND $\text{finish}[v] < \text{finish}[u]$
- **Cross Edge:** everything else

Tasks

1. Implement `dfs_timestamps(graph, 0)` returning `disc[]` and `finish[]`
2. For each edge (u,v) , classify using the rules above
3. Confirm: $(4 \rightarrow 1)$ is a back edge (cycle in directed graph)
4. Confirm: $(1 \rightarrow 2)$ is a cross edge (2 already discovered via $0 \rightarrow 2$ before $1 \rightarrow 2$ explored)

✅ **Expected result:** $(4 \rightarrow 1)$ is a back edge confirming a directed cycle. $(1 \rightarrow 2)$ is a cross edge because vertex 2 was already fully discovered via the path $0 \rightarrow 2$ before the edge $1 \rightarrow 2$ was explored.

Hands-On Lab 2: Tarjan's SCC & Condensation DAG

ADVANCED LAB

Graph

```
0 → 1 → 2 → 0 (SCC: {0,1,2})
2 → 3
3 → 4 → 3    (SCC: {3,4})
4 → 5        (SCC: {5})
```

Tasks

1. Implement Tarjan's SCC; find the 3 SCCs above
2. Build the condensation DAG (each SCC → single vertex)
3. Find topological order of the condensation DAG
4. Verify: condensation of any directed graph is always a DAG

Condensation DAG

```
SCC0 = {0,1,2}
SCC1 = {3,4}
SCC2 = {5}
```

Condensation:

```
SCC0 → SCC1 → SCC2
```

(A DAG — condensation is ALWAYS a DAG)

Topological order: SCC0, SCC1, SCC2

Production insight: Building the condensation DAG is how many network analysis systems simplify complex directed graphs — replace strongly connected clusters with single super-nodes for higher-level analysis.

DFS Performance Considerations

DFS Variant	Time	Space	Notes
Standard DFS	$O(V+E)$	$O(V)$	Call stack depth = max path length
DFS with timestamps	$O(V+E)$	$O(V)$	disc + finish arrays
Tarjan's SCC	$O(V+E)$	$O(V)$	Single pass
Kosaraju's SCC	$O(V+E)$	$O(V)$	Two passes; builds reversed graph
Bridge/AP detection	$O(V+E)$	$O(V)$	Single pass with low[]
Backtracking DFS	$O(b^d)$ worst	$O(d)$	b=branching, d=depth; pruning critical
DFS + memoization	$O(V+E)$	$O(V)$	Only valid on DAGs

Recursion Stack Overflow — Production Risk

Default call stack limits:

- Python: ~1,000 frames (sys.getrecursionlimit())
- Java: ~10,000–100,000 frames (JVM-dependent)
- C/C++: ~100,000+ frames

For a path graph of $n=100,000$ nodes:

- Recursive DFS: 100,000 frames → stack overflow in Python/Java
- Iterative DFS: explicit stack in heap → no overflow, handles any depth

Production rule:

- $n \leq 10,000$: recursive DFS is fine
- $n > 10,000$ or unknown depth: iterative DFS required

DFS Best Practices & Common Mistakes

✓ DO'S

Category	Practice
Design	Choose DFS for: cycle detection, SCC, topo sort, bridges, backtracking
Implementation	For directed cycle detection: use 3-state (NOT, IN_PROGRESS, DONE)
Implementation	For Tarjan's: always initialize <code>low[u] = disc[u]</code> at discovery
Performance	For $n > 10,000$: use iterative DFS to avoid stack overflow
Correctness	In backtracking: always restore state after recursion

✗ DON'TS

Category	Anti-Pattern
Design	Don't use DFS for unweighted shortest path — use BFS
Correctness	Don't use only <code>visited[]</code> for directed cycle detection
Correctness	In Tarjan's: don't update <code>low[u]</code> from a vertex NOT on the current stack
Performance	Don't apply recursive DFS on large, potentially deep graphs
Backtracking	Don't forget to undo state changes on backtrack

Level	Mistake	Impact	Fix
Intermediate	Using <code>visited[]</code> alone for directed cycle	Misses directed cycles	Use <code>rec_stack[]</code> or 3-state
Intermediate	Forgetting to restore state in backtracking	Wrong results for next path	Always <code>undo(action)</code> after recursive call
Advanced	In Tarjan's: updating <code>low[u]</code> from non-stack visited node	Wrong SCCs	Only update when <code>on_stack[v]</code> is TRUE
Advanced	Memoization on cyclic graph	Infinite recursion	Memoized DFS only valid on DAGs
Advanced	Not resetting <code>rec_stack</code> on backtrack	Treats non-cycle edges as cycles	Always <code>rec_stack[u] = FALSE</code> before returning

BFS vs DFS — Comprehensive Comparison

Dimension	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO) — call stack or explicit
Exploration Order	Level by level (wide)	Branch by branch (deep)
Shortest Path	✔ Guaranteed (unweighted)	✘ Not guaranteed
Cycle Detection (directed)	✔ (with rec_stack via BFS)	✔ (natural with rec_stack)
SCC Detection	✘	✔ Tarjan's / Kosaraju's
Topological Sort	✔ Kahn's algorithm	✔ DFS finish order
Bridges / Art. Points	✘	✔ DFS + low[]
Backtracking	✘	✔ Natural with recursion
Memory (balanced tree)	$O(\text{width})$ — can be $O(n/2)$	$O(\text{height})$ — $O(\log n)$ balanced
Stack Overflow Risk	✘ (iterative)	✔ (recursive)
Bipartite Check	✔ Elegant with BFS coloring	✔ Also works
Connected Components	✔	✔ Both work equally

📌 **Rule of Thumb:** Use BFS when the question is about **DISTANCE or LEVELS**. Use DFS when the question is about **STRUCTURE** (cycles, SCCs, bridges, paths, backtracking). When in doubt for graph traversal: BFS is safer (no stack overflow, guarantees shortest path).

DFS Learning Summary

1. LIFO Stack

DFS uses a LIFO stack (call stack); explores deep before wide

2. Edge Types

DFS tree with edge types: tree, back, forward, cross — these underlie all advanced applications

3. disc[] and low[]

Core tools: discovery time and minimum reachable discovery time via back edges

4. Cycle Detection

3-state DFS (NOT/IN_PROGRESS/DONE) or visited + rec_stack for directed graphs

5. Topological Sort

DFS finish order reversed; equivalent to Kahn's BFS approach

6. Tarjan's SCC

Single DFS pass; SCC root identified when `low[u] == disc[u]`

7. Bridges & APs

Bridge: `low[v] > disc[u]`. Articulation point: `low[v] >= disc[u]`

8. Backtracking

DFS with state restoration — always undo on return. For production: use iterative DFS

DFS QUICK REFERENCE

STRUCTURE: LIFO Stack (recursive = call stack)
GUARANTEE: Complete traversal; NOT shortest path
TIME: $O(V + E)$ | SPACE: $O(V)$

EDGE TYPES (directed DFS):
Tree: v unvisited at traversal time
Back: v in current rec_stack $\rightarrow$ CYCLE in directed graph
Forward: v is descendant, not direct tree child
Cross: v in different DFS subtree

KEY ARRAYS:
disc[u]: when u was first visited
low[u]: min disc reachable from u 's subtree via ≤ 1 back edge

APPLICATIONS:
SCC (Tarjan): `low[u] == disc[u]` $\rightarrow$ SCC root
Bridge: `low[v] > disc[u]` for tree edge (u,v)
Articulation: `low[v] >= disc[u]` for tree edge (u,v)
Topo Sort: reverse of DFS finish order
Backtracking: recurse + undo state on return

Common Interview & Competitive Programming Questions

BFS QUESTIONS

01

Shortest Path

Find shortest path between two vertices in an unweighted graph.

02

BFS vs Dijkstra

What is the difference? When does BFS suffice?

03

Multi-Source BFS

Implement multi-source BFS. Give two real-world examples.

04

Bidirectional BFS

How does bidirectional BFS reduce time complexity?

05

Implicit Graph BFS

Implement BFS on an implicit graph (word ladder, knight moves).

06

0-1 BFS

Explain 0-1 BFS. Why is a deque used instead of a queue?

DFS QUESTIONS

01

Directed Cycle Detection

Detect cycle in a directed graph. Why is `visited[]` alone insufficient?

02

Tarjan's SCC

Explain Tarjan's SCC algorithm. What is `low[u]`?

03

Bridges & APs

What is a bridge? An articulation point? How do you find them?

04

Topological Sort

How does DFS-based topological sort work? Why reverse finish order?

05

Tarjan vs Kosaraju

Compare Tarjan's and Kosaraju's SCC algorithms.

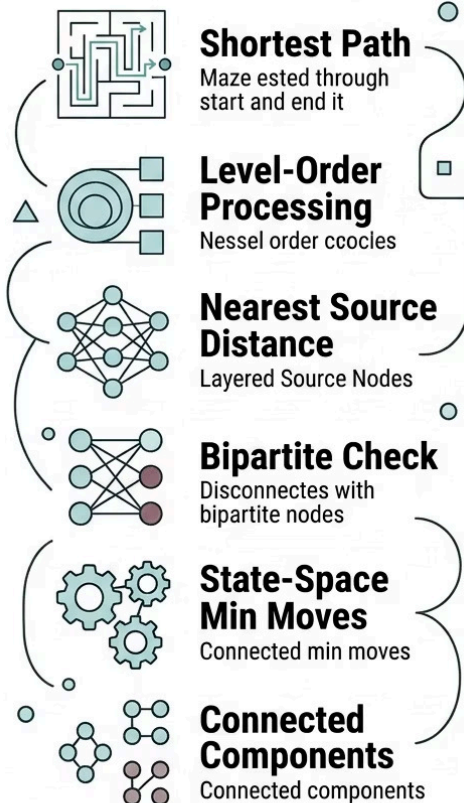
06

N-Queens Backtracking

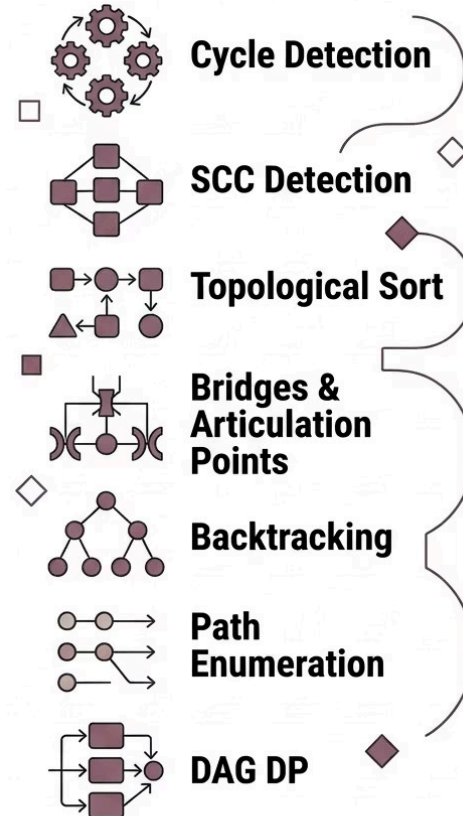
Implement backtracking DFS for N-Queens. How does backtracking work?

BFS vs DFS — When to Choose Which

BFS APPLICATIONS:



DFS APPLICATIONS:



The Golden Rule

Use BFS when the question is about **DISTANCE** or **LEVELS**.
Use DFS when the question is about **STRUCTURE**.

Space Complexity Nuance

- **Balanced binary tree:** BFS uses $O(n/2)$ queue; DFS uses $O(\log n)$ stack → DFS wins
- **Path graph (linked list):** BFS uses $O(1)$ queue; DFS uses $O(n)$ stack → BFS wins
- **General graphs:** Both $O(V)$ — equivalent

Can DFS Find Shortest Path?

Only on trees (where there is exactly one path between any two vertices). On general graphs, DFS does not guarantee shortest path — use BFS for unweighted, Dijkstra for weighted.

Recommended Next Topics



Dijkstra's Algorithm

BFS with priority queue for weighted SSSP. The natural extension of BFS to weighted graphs.



Bellman-Ford

SSSP with negative weights; detects negative cycles. Handles cases Dijkstra cannot.



Floyd-Warshall

All-pairs shortest path via DP. $O(V^3)$ but solves all-pairs in one pass.



Minimum Spanning Tree

Prim's (BFS-like) and Kruskal's (Union-Find). Foundational for network design.



Union-Find (DSU)

$O(\alpha(n))$ component queries; Kruskal's backbone. Essential for dynamic connectivity.



Network Flow

Ford-Fulkerson / Edmonds-Karp. Max-flow / min-cut. BFS at its core (Edmonds-Karp).



A* Search

BFS with heuristic for game pathfinding. Combines BFS optimality with DFS efficiency.



Graph DP

Longest path on DAG, counting paths, DP on tree. DFS + memoization at scale.